PORTFOLIO OF EVIDENCE

SmartX

Implementation report. Simulated IoT gateway for hydroponics, utilities, and site-meter telemetry.

Alton Kholwani Muganda
Student number ST10263456
Module: PROG7312
POE

Contents

1. Executive summary 3
2. Project arc — Part 1 to Final PoE 4
2.1 Part 1 — Telemetry 4
2.2 Part 2 — Command stream 4
2.3 Part 3 — Mesh routing 5
3. Architecture of the unified system 6
4. How to run the completed system 7
5. Feedback-driven improvements 9
6. Task 3 — architectural and algorithmic challenges 10
6.1 Shared contracts 10
6.2 AVL history tree 10
6.3 Modelling the mesh 11
6.4 Traversal semantics 11
6.5 Prim’s MST 11
6.6 Keeping Parts 1 and 2 alive 12
7. Key learnings 13
7.1 Data structures 13
7.2 Algorithms 13
7.3 Async UI and background work 14
7.4 Integration discipline 14
7.5 A larger deployment 14
8. Recommended additional technologies 15
8.1 Cloud-hosted database 15
8.2 GitHub Actions 15
8.3 LLM-assisted alert summaries 15
8.4 Home Assistant 16
8.5 Containerisation 16
8.6 Observability 16
9. Conclusion 18
Appendix A — Key source map 19
Bibliography 20

1. Executive summary

I built Smart-X as a simulated South African IoT gateway for hydroponics, utilities, and site-meter readings. The devices are simulated ESP32s (Espressif Systems, n.d.). They send mixed packets: environmental floats, power integers, and actuator on/off values. An ASP.NET Core API checks and stores those packets (Microsoft, 2025i). I look at them in a Blazor WebAssembly console (Microsoft, 2025b).

This is one app with three parts, not three separate demos. I keep everything in one `GatewayStore` singleton for the life of the process (Microsoft, 2025c). One boot seeds the fleet, the faults, the queues, and the mesh. Restarting the API is enough to run the full demo again.

Pillar	Focus	Operator entry
Part 1	Sensor registration, typed ingest, Progressive Disclosure, health and freshness	`/telemetry`
Part 2	Real-time command stream: queues, undo stack, timeline, HashSet alerts, suggestions	`/commands`
Part 3	AVL history trees, mesh graph, BFS reachability, Prim MST, topology dashboard	`/mesh`

2. Project arc — Part 1 to Final PoE

2.1 Part 1 — Available Sensor Data Ingestion and Telemetry

What I wanted. I needed to take in mixed telemetry and let an operator look at the fleet in steps: overview, then filters, then one device, then its history. Silent and faulty devices stay on the list. I used progressive disclosure so the first screen stays readable and the extra detail is one click away (Nielsen, 2006).

What I built:

- A nested deployment tree: site, zone, sub-zone, then node, checked recursively. A tree fits because each place contains smaller places (Cormen et al., 2022).
- Registration and typed ingest for environmental, power, and actuator packets. I store them in `List<T>` collections (Microsoft, 2025g).
- Health and freshness bands: live, aging, stale, and disconnected.
- Seeded faults: a temperature spike, a stuck actuator, a silent sensor, bad frames, a location outage, and a recovery.
- Multipart diagnostic attachments.
- A Blazor investigation screen at levels L0 to L3 (Microsoft, 2025b).

Structures I used. Lists, dictionaries to find a device, and a recursive check of the deployment tree. The .NET `Dictionary<TKey,TValue>` is a hash table, so a lookup does not walk the whole fleet (Microsoft, 2025d). With the usual hashing assumptions, that lookup is expected constant time (Cormen et al., 2022).

2.2 Part 2 — Real-Time Command Stream and History

What I wanted. Critical alerts should jump ahead of the normal FIFO backlog. An operator should be able to override an actuator and undo that override. I also wanted unique disconnect and error keys, and suggestions based on what the operator has already done.

What I built:

- A normal FIFO ingest queue, using `Queue<T>` (Microsoft, 2025l), next to a critical priority queue. `PriorityQueue<TElement,TPriority>` takes the smallest priority key first (Microsoft, 2025k). My key is severity, then the earlier `RaisedAt`.
- A background `TelemetryQueueConsumer`. It is an ASP.NET Core hosted `BackgroundService` (Microsoft, 2026c). The request only enqueues.
- A severity rank, with `RaisedAt` as the tie-breaker.
- An undo stack, `Stack<T>` (Microsoft, 2025n), and a `SortedDictionary` timeline. Entries come out in key order, so I do not sort a list again on every read (Microsoft, 2025m).
- `HashSet` keys for disconnects and errors. A duplicate key is rejected (Microsoft, 2025e).
- A recommendation engine. Confidence is a score between 0 and 1.
- A `/commands` page with filters, issue and undo, suggestions, and a five-second refresh from `PeriodicTimer` (Microsoft, 2025j).

After the Part 2 feedback I changed several things before and during Part 3. Filters now survive navigation. The menu is grouped by pillar. The command page refreshes itself. A background worker drains the queues. Equal-severity alerts use a time tie-break. Disconnects are detected. Suggestions use a confidence score instead of a hard-coded moisture rule. I logged that work in `docs/FEEDBACK-UPDATES.md`.

2.3 Part 3 — Network Topology and Mesh Routing

What I wanted. I needed a radio mesh, a fast way to query old readings, a walk that shows who can still reach the gateway, and a cheap routing backbone. A wireless mesh is a multi-hop network: nodes forward for each other, and extra links give another path when one fails (Akyildiz, Wang and Wang, 2005). I simulated that shape. I did not implement an 802.11s or 802.15.4 stack.

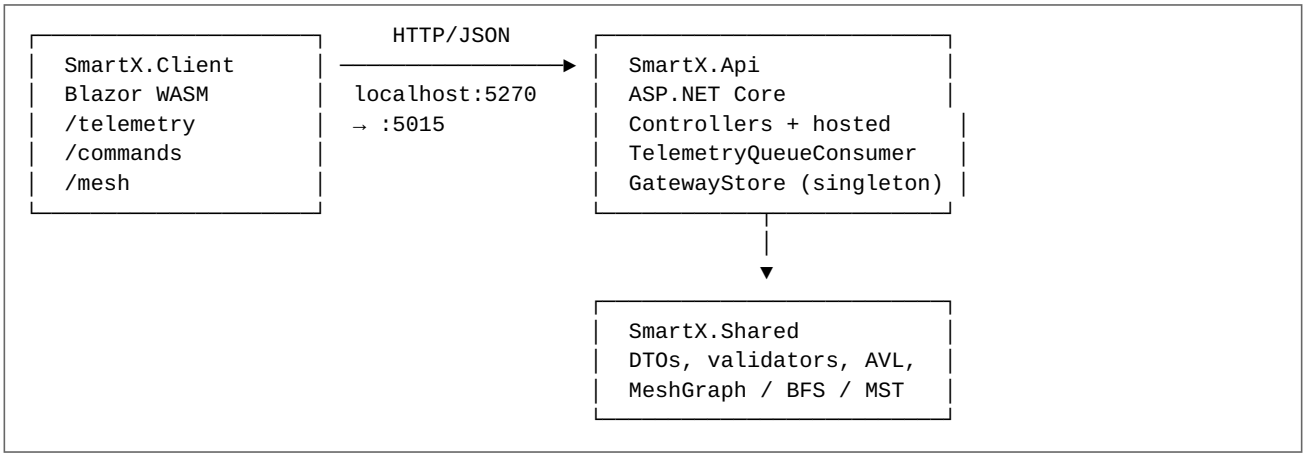
Delivered:

Structure or algorithm	Role	Evidence
AVL trees (per device)	Timestamp range history without a full list scan, about O(log n + k)	`AvlTelemetryHistoryTree`, `GET …/history/range`, `/mesh` historical log
Undirected mesh graph	Gateway plus 16 ESP32s, at least 20 weighted wireless links	`MeshGraph`, `MeshTopologySeeder`, `GET /api/mesh/topology`
BFS and DFS	Live reachability, down versus cut-off, alternate hops	`MeshGraphTraverser`, `MeshReachabilityAnalyzer`, `GET /api/mesh/traversal`
Prim’s MST	Minimum-cost backbone and parent hops toward the gateway	`MeshMstBuilder`, `GET /api/mesh/mst` (about 25 links down to 16, about 47% cost saved)
Topology UI	SVG mesh, route track by id or MAC, operator briefing, AVL log	`/mesh`

An AVL tree is a height-balanced binary search tree. After each insert, the two child subtrees of any node differ in height by at most one (Adelson-Velsky and Landis, 1962). On a binary search tree, search and range walks take time proportional to the height. Balance keeps that height logarithmic in the number of nodes (Cormen et al., 2022). Breadth-first and depth-first search are the usual graph walks. Breadth-first search gives hop distance because it expands one level at a time (Cormen et al., 2022; Sedgewick and Wayne, 2011). Prim’s algorithm grows a minimum spanning tree from a chosen root. Each step adds the cheapest edge that leaves the tree (Prim, 1957).

Seeded mesh fault. `MeshDownScenarioSeeder` forces `sx-pwr-inverter` offline. I can then show a healing hop next to the silent sump disconnect from Parts 1 and 2.

3. Architecture of the unified system



The Blazor WebAssembly client calls the API with `HttpClient` (Microsoft, 2026d). I made the API and the client agree on camelCase names and string enums, using `System.Text.Json` and `JsonStringEnumConverter` (Microsoft, 2026g; Microsoft, 2025f). The browser on `http://localhost:5270` is allowed to call `http://localhost:5015` by the API's CORS policy (Microsoft, 2026f).

I can also start both with Docker Compose, on the same host ports (Docker, n.d.-a). There is no database. Restarting the API clears the in-memory store.

4. How to run the completed system

I run this from a clone of the repository. The three projects target `net10.0`, so I need the .NET 10 SDK. Docker is optional. There is no test project in the repo, so there is nothing to run with `dotnet test`.

What I need

- .NET 10 SDK, for `dotnet run` (Microsoft, 2026e).
- Optional: Docker with Compose v2, if I want both services in containers instead of two terminals (Docker, n.d.-a).

Clone, restore, and build

```
git clone https://github.com/akzz98/SmartX.git
cd SmartX
dotnet restore SmartX.slnx
dotnet build SmartX.slnx
```

`SmartX.slnx` builds `SmartX.Shared`, `SmartX.Api`, and `SmartX.Client`.

Config

The client reads `ApiBaseUrl` from `SmartX.Client/wwwroot/appsettings.json`. Mine is `http://localhost:5015/`. The API allows the browser origin `http://localhost:5270` in `SmartX.Api/appsettings.Development.json` under `Cors:Origins`. I use the `http` launch profiles so that origin matches. If I serve the client from another origin, I add it to `Cors:Origins` and I change `ApiBaseUrl`.

Run the API, then the client

I start the API first. If only the client is running, the status strip says the gateway is unreachable.

Terminal 1:

```
dotnet run --project SmartX.Api --launch-profile http
```

That listens on `http://localhost:5015`.

Terminal 2:

```
dotnet run --project SmartX.Client --launch-profile http
```

That listens on `http://localhost:5270`. I open `http://localhost:5270`. The console should show Gateway Online, 16 devices, and all three pillars Available. From there I use `/telemetry`, `/commands`, and `/mesh`.

A quick check of the API:

```
curl http://localhost:5015/api/sensors/summary
curl http://localhost:5015/api/stream/snapshot
curl http://localhost:5015/api/mesh/topology
curl http://localhost:5015/api/mesh/traversal
curl http://localhost:5015/api/mesh/mst
```

Docker Compose, instead of two terminals

I do not run Compose and `dotnet run` at the same time. Both want ports 5015 and 5270. From the repository root:

```
docker compose up --build
```

The API container maps host port 5015 to port 8080. The client container maps host port 5270 to nginx on port 80. Compose sets `ASPNETCORE_ENVIRONMENT=Docker` and `Cors__Origins__0=http://localhost:5270`. CORS for that origin is also in `SmartX.Api/appsettings.Docker.json`. The browser still uses `ApiBaseUrl` (`http://localhost:5015/`), which is the host port, not the Docker service name. I open `http://localhost:5270` as before.

Stop

In each `dotnet run` terminal I press Ctrl+C. For Compose I press Ctrl+C, or I run `docker compose down` from the repository root. Restarting the API clears the in-memory store and loads the seeds again.

5. Feedback-driven improvements (Parts 1–2 → Final PoE)

I closed the Part 2 feedback before and during the Part 3 work, and I logged it in `docs/FEEDBACK-UPDATES.md`. Those fixes are still in the same app.

- I changed the filters so they survive navigation. They live in a Blazor singleton, not in fields the router throws away with the page (Microsoft, 2025c).
- I grouped the menu by pillar. It is no longer a flat list of pages (Nielsen, 2006).
- I made the command stream refresh itself with a `PeriodicTimer` (Microsoft, 2025j).
- I added a background consumer to drain the queues. Ingest no longer drains them on the request (Microsoft, 2026c).
- I break priority ties on time, using the comparison key of `PriorityQueue<TElement,TPriority>` (Microsoft, 2025k).
- I detect disconnects from freshness and store them as `HashSet` keys, so the same disconnect is not raised twice (Microsoft, 2025e).
- I score suggestions with a confidence between 0 and 1. I removed the hard-coded moisture-to-pump rule.

The earlier parts still run. That was a Final PoE requirement, and these changes keep them working.

6. Task 3 — architectural and algorithmic challenges (and how they were overcome)

Final PoE Task 3 was network topology and mesh routing. I did not start a new service for it. I had to add it inside the Part 1 and Part 2 gateway without breaking progressive disclosure or the command stream (Nielsen, 2006). The problems below are the ones I actually hit in this code.

6.1 Shared contracts across API and Blazor WASM

The problem. The Blazor client has to show topology, traversal, the MST, and AVL range results without me rewriting the graph code in JavaScript. Enums such as `FreshnessState` and `MeshNodeKind`, and nested DTOs, break if the JSON shape drifts (Microsoft, 2026g).

What I did.

- I put the mesh and AVL models and DTOs in `SmartX.Shared`. Both the API and the client reference that project.
- I made both sides use camelCase names and string enums (`JsonStringEnumConverter`) (Microsoft, 2025f).
- The client uses one `HttpClient`, `ApiBaseUrl` in `wwwroot/appsettings.json`, and `GatewayJson.Options` when it reads JSON (Microsoft, 2026d).
- CORS lets `http://localhost:5270` call `http://localhost:5015`. The Docker settings in `appsettings.Docker.json` do the same (Microsoft, 2026f).

How to see it. `MeshController` returns `MeshTopologySnapshotResponse`, `MeshTraversalResponse`, and `MeshMstResponse`. The `/mesh` page binds those same types.

6.2 Choosing and integrating an AVL history tree (not a list scan)

The problem. Part 1 already kept packets in lists for the history screens (Microsoft, 2025g). Task 3 asked for range queries, such as moisture for one device between two times, that stay fast as the history grows. A full list scan would fail the “trees for historical queries” part of the brief, and it would look the same as Part 1.

What I did.

- I added a per-device AVL tree, `AvlTelemetryHistoryTree`. The key is timestamp, then sequence. `TelemetryHistoryIndex` holds the trees (Adelson-Velsky and Landis, 1962).
- Every accepted ingest writes the list and the tree on the same path.
- `QueryRange` skips subtrees outside `[from, to]`. On a binary search tree the walk only enters a subtree that can overlap the window, so the cost follows the height plus the number of matches (Cormen et al., 2022). The API returns `nodesVisited` and `treeSize` so the screen can show that.
- The `/mesh` history log and `GET /api/sensors/{id}/history/range` use that same path.

How to see it. The demo device is `sx-env-rack1-moist`, metric `Moisture`. The range result shows how many samples matched and how many tree nodes were visited.

6.3 Modelling a realistic mesh on top of an existing fleet

The problem. I could not use a toy triangle. The graph had to use the seeded 16 ESP32s (Espressif Systems, n.d.), include a gateway node, give devices more than one neighbour, and put a non-negative weight on each edge for the MST (a power or RSSI-style cost). Online and offline had to follow the same freshness as Parts 1 and 2.

What I did.

- `MeshGraph` is an adjacency list. Nodes are the gateway plus the sensors. Edges are undirected and weighted. An adjacency list fits when the algorithm walks neighbours instead of scanning a matrix (Sedgewick and Wayne, 2011).
- `MeshTopologySeeder` builds at least 20 links: rack clusters, the feeder bay, and a backbone between racks. The extra links are what give a mesh another path (Akyildiz, Wang and Wang, 2005).
- `RefreshOnlineStatus` sets each node from `LastSeenAt`. A Part 1 or Part 2 disconnect, such as the silent sump, shows up as mesh-down.
- `MeshDownScenarioSeeder` ages `sx-pwr-inverter`, so the BFS healing demo is there after every boot.

How to see it. `GET /api/mesh/topology` returns 17 nodes and 25 edges. The offline count includes the inverter and the sump.

6.4 Traversal semantics: topology vs live routing

The problem. One BFS over every edge cannot answer both “what was the path if everything was up?” and “who can still forward now?”. The operator also needs another hop when a relay is down, not only the visit order (Cormen et al., 2022).

What I did.

- `MeshGraphTraverser` does BFS and DFS. `onlineOnly` stops the walk from stepping through an offline neighbour (Sedgewick and Wayne, 2011).
- `MeshReachabilityAnalyzer` runs a topology BFS and a live BFS. It then marks each device reachable, down, or cut-off. Cut-off means the device is on, but a failed relay has stranded it.
- For each down node I suggest the live neighbour closest, by hop count, to the gateway. I also keep the ideal path next to that alternate.
- The API is `GET /api/mesh/traversal`. The screen is the operator briefing on `/mesh`. That HTTP result is BFS, because hop count is what I use for routing. DFS is still there on `MeshGraphTraverser.DepthFirst`.

How to see it. The traversal lists `sx-pwr-inverter` and `sx-env-sump-temp` as down, with a suggested next hop, usually `sx-gateway`.

6.5 MST choice (Prim) and comparing it to the full mesh

The problem. The rubric accepts Prim or Kruskal. I needed a routing tree rooted at the gateway, with a parent hop for each device, and a clear comparison of the full mesh against that cheaper backbone.

What I did.

- `MeshMstBuilder` uses Prim's algorithm (Prim, 1957). Kruskal's algorithm also builds a minimum spanning tree, by sorting every edge and joining components (Kruskal, 1956). I chose Prim because the tree starts at the gateway, the graph is already an adjacency list, and each node's parent comes out of that growth (Sedgewick and Wayne, 2011).
- `onlineOnly=true` is what `/mesh` uses when it tracks a route. The full MST payload includes `MeshVsMstComparison`: edges saved, cost saved, and the links I dropped.
- The seeded numbers stay the same each boot: about 25 links at cost 42.7, against 16 links at cost 22.7. That is about 47 percent of the radio cost saved.

How to see it. `GET /api/mesh/mst` returns the comparison. The SVG highlights the path from `sx-env-rack1-moist` to `sx-gateway`.

6.6 Keeping Part 1 and Part 2 alive under Part 3 load

The problem. The mesh APIs and the `/mesh` page could have starved the queue consumer, frozen the screen, or turned Part 3 into a separate app.

What I did.

- Mesh reads refresh status under the existing `GatewayStore` lock. They do not replace ingest or the queues.
- The hosted `TelemetryQueueConsumer` still drains priority, then FIFO, even while topology GETs are running (Microsoft, 2026c).
- `/mesh` loads topology, the MST, traversal, and history asynchronously. If the API is down, empty, or in error, the page says so (Microsoft, 2025b).
- The startup menu and the sidebar still show all three pillars. The README explains how to demo all three from one boot.

How to see it. After I restart the API, the summary, the stream snapshot and suggestions, and the mesh topology, traversal, and MST all succeed together.

7. Key learnings (linked to actual Smart-X tasks)

These are things I learned while building Parts 1 to 3 in one gateway. They come from this code, not from a textbook on its own.

7.1 Data structures — pick the structure that matches the operator question

Question I had to answer	What I used	Why I chose it
Which device is this id or MAC?	Dictionaries in `SensorFleet`	Expected O(1) lookup on ingest and command (Microsoft, 2025d; Cormen et al., 2022)
What happened to overrides and alerts over time?	`SortedDictionary` timeline	Ordered history, without sorting a list again on each GET (Microsoft, 2025m)
Has this disconnect already been raised?	`HashSet` unique keys	A duplicate is dropped (Microsoft, 2025e)
Moisture for rack 1 between T1 and T2?	Per-device AVL tree	Range walk about log n + k; `nodesVisited` shows it (Adelson-Velsky and Landis, 1962)
Who is linked on the radio mesh?	Adjacency-list `MeshGraph`	Neighbour scans, BFS, and Prim (Sedgewick and Wayne, 2011)
Cheapest backbone to the gateway?	Prim MST parent map	Gateway-rooted hops for `/mesh` (Prim, 1957)
Critical alert versus FIFO backlog?	Priority queue plus FIFO queue	Severity, then time, versus arrival order (Microsoft, 2025k; Microsoft, 2025l)

What I took from this. I picked each structure for the question the screen or the API has to answer, then seeded it so the demo does not change between boots. Swapping a list for an AVL was worth it because ingest writes both, and the range API shows the visit count.

7.2 Algorithms — mesh theory applied to a hydroponics gateway

- **BFS versus DFS.** BFS from `sx-gateway` gives hop distance and a healing path. I kept DFS for a deep walk. I report reachability with BFS because the useful question is how many hops sit between a device and a live neighbour (Cormen et al., 2022).
- **Online-only walks.** A mesh can fail in part, not only all at once (Akyildiz, Wang and Wang, 2005). I run a topology BFS next to a live BFS. That separates down (offline) from cut-off (online, but stranded). Both show up in `GET /api/mesh/traversal`.
- **MST as a radio budget.** The full mesh, about 25 links, is the resilient graph. Prim's tree, about 16 links, is the one I would use day to day (Prim, 1957). The comparison says about 47 percent of the radio cost is saved.
- **Why Prim, not Kruskal.** Readings route toward one fixed gateway. Growing the tree from that root gives me parent hops without a second pass to root the tree (Kruskal, 1956; Sedgewick and Wayne, 2011).

What I took from this. The mesh ideas only become clear when I tie them to seeded devices (`sx-pwr-inverter` down, the moist-sensor path highlighted) and to plain wording on `/mesh`.

7.3 Async UI and background work — do not block the operator

- The Part 2 feedback showed me the cost of draining the queue inside the request. Ingest now only enqueues. `TelemetryQueueConsumer` drains priority, then FIFO (Microsoft, 2026c).
- `PeriodicTimer` on `/commands` (five seconds) refreshes the figures and suggestions without a manual reload. It pauses during override and undo so my click is not overwritten (Microsoft, 2025j).
- I needed `OperatorSessionState` because a Blazor page is a component. Filters kept only on that component disappear when I navigate (Microsoft, 2025c).
- `/mesh` loads topology, traversal, the MST, and AVL history asynchronously, and it says so if something fails. A dead port 5015 is obvious instead of a spinner that never ends (Microsoft, 2025b).

What I took from this. The screen has to stay usable while the gateway works. Async Blazor plus a hosted consumer keeps the operator’s wait separate from queue processing.

7.4 Integration discipline — one store, three pillars

- Sharing `GatewayStore` forced freshness, alerts, and mesh online flags to stay the same fact. The silent sump and the mesh-down inverter are that same fact.
- `SmartX.Shared` is the contract, so I did not copy the graph and AVL code into the browser (Microsoft, 2026d).
- When I added Docker Compose I kept the same ports and the same CORS setup. I did not design a second system for containers (Docker, n.d.-a).

What I took from this. Building Part 1, then Part 2, then Part 3 in one process was harder than three separate demos. It matches the brief, and the repository tells one story.

7.5 What I would watch in a larger deployment

What I would add is durable storage, login, and more than one instance. I would still keep the same structures: indexed history, a graph snapshot, and an MST recomputed when the topology changes.

8. Recommended additional technologies

Smart-X already runs on my machine, and I can also start it with Docker Compose (Docker, n.d.-a). The tools below are what I would add for a real site. Each one fills a gap I hit in Parts 1 to 3. I judged them on security, scale, speed, and how well they fit .NET.

8.1 Cloud-hosted database (Azure SQL / Cosmos DB / PostgreSQL on Azure)

What I would replace. The in-memory `GatewayStore`. It disappears every time the API restarts.

What I would add. Stored sensors, packet history, commands, alerts, and mesh snapshots.

Why. Restarting the API clears the in-memory store. A hydroponics site that must keep history after a reboot, and across more than one API instance, needs a database. Range queries would then read an index instead of scanning process memory.

How it fits .NET. EF Core and the Azure Cosmos DB SDK plug into the ASP.NET Core services I already use, without leaving C# (Microsoft, 2020; Microsoft, 2026h). Azure SQL Database is the managed relational option on the same platform (Microsoft, 2026i). For security I would use managed identity and Azure Key Vault, so the connection string is not in `appsettings`, and database roles instead of one shared administrator password (Microsoft, 2025h; Microsoft, 2026b). Several API copies can share one store. A Cosmos partition key such as `sensorId` or site matches a growing fleet (Microsoft, 2026h). Indexed range queries replace a linear scan of history, while the hot AVL and mesh graphs can stay in memory and write through. I would still use the same `dotnet` tools.

8.2 GitHub Actions CI/CD

What I would replace. A manual `dotnet build`, and “it works on my machine”, before I push.

What I would add. Workflows on `push` and `pull_request` that restore, build the API and the client, optionally curl a test host, and publish Docker images (GitHub, n.d.; Microsoft, 2026e).

Why. The `POE` branch already has a long history. CI would catch a mesh or queue break when a feedback fix and Part 3 land together.

How it fits .NET. The official setup actions and `dotnet restore`, `build`, `test`, and `publish` are built for this, and a static Blazor publish fits the same job list as the API (Microsoft, 2026e). OIDC to a registry means I do not keep a long-lived deploy secret. The API and client jobs can run in parallel, and NuGet can be cached. The build should fail on compile or test before a Docker layer spends minutes (Docker, n.d.-b). Pull-request checks on `POE` and `main` match the GitHub remote I already use (`akzz98/SmartX`).

8.3 LLM-assisted alert summaries (Azure OpenAI / GitHub Models API)

What I would replace. An operator reading raw `HashSet` keys and long alert reasons on `/commands` (Microsoft, 2025e).

What I would add. An endpoint that turns recent priority alerts and disconnect keys into a short brief. For example: irrigation risk on rack 1, inverter relay down, use the gateway hop.

Why. Part 2 suggestions already score patterns. A language model could describe the current incident. I would not let it replace the fixed recommendation engine.

How it fits .NET. Semantic Kernel can sit as a scoped service next to the controllers I already have (Microsoft, 2023). Azure OpenAI, now described in Microsoft's Foundry model docs, is the hosted model that brief would call (Microsoft, 2026j). I would keep prompts on the server, not send actuator secrets, and limit the rate so the WebAssembly client cannot spend the quota. Summary jobs can use the same priority-queue pattern, so ingest stays fast (Microsoft, 2025k). I would call the model only on an alert burst or when the operator asks, and cache the brief against a hash of the disconnect set. A flag in `appsettings` would turn summaries on without a second Blazor UI.

8.4 Home Assistant (MQTT / REST integration)

What I would replace. Simulated ESP32 traffic only (Espressif Systems, n.d.).

What I would add. A bridge from Smart-X commands and readings to Home Assistant devices — valves, pumps, moisture sensors — over MQTT or the Home Assistant REST API (Home Assistant, n.d.-a; Home Assistant, n.d.-b).

Why. That is closer to a real site. Part 2 overrides such as `OpenValve` and `StartPump` could drive a real or virtual actuator. Progressive disclosure would still be how I inspect gateway state (Nielsen, 2006).

How it fits .NET. MQTTnet is a .NET MQTT client and server library, and `HttpClient` is already how the Blazor client calls the API (MQTTnet, n.d.; Microsoft, 2026d). MQTT is the OASIS publish/subscribe protocol used for device readings (OASIS, 2019). A hosted `BackgroundService` can bridge Home Assistant and `GatewayStore` the same way today's queue consumer does (Microsoft, 2026c). I would keep long-lived tokens in a secret store, use TLS for MQTT, and check an override before publishing it. MQTT keeps bursty device traffic off the API threads, and each site can be its own topic. Publish and subscribe would be asynchronous so ingest is not blocked. A local Home Assistant plus the Docker Compose stack I already have can show the bridge before any cloud IoT hub (Docker, n.d.-a).

8.5 Deeper containerisation / orchestration (already started)

What I have now. `docker compose up --build` runs the API and an nginx client for the WebAssembly app (Docker, n.d.-a; nginx, n.d.).

What I would add. Azure Container Apps or Kubernetes, for more than one copy, health checks, and separate scaling of the API and the static client. I would also put `App_Data` attachments on a volume or in a database (Microsoft, 2026a; Kubernetes, n.d.).

Why. Compose shows that the app can be packaged. I would want orchestration once a cloud database and CI are publishing images all the time.

How it fits .NET. The Dockerfiles I already wrote are multi-stage and use the official SDK and ASP.NET runtime images (Docker, n.d.-b). Container Apps can call the platform health endpoints (Microsoft, 2026a). I would run non-root containers, use a private registry, and use managed identity to reach the database (Microsoft, 2025h). The API can scale separately from the static nginx client, and later it could scale with queue depth. Layer caching, and a trimmed or ReadyToRun publish, help cold starts. The same Compose files can move toward Container Apps or Kubernetes with little Dockerfile change, and CI can publish the images those workflows already build (GitHub, n.d.).

8.6 Observability (OpenTelemetry + Application Insights or Grafana)

What I would replace. Console logs and manual curl checks.

What I would add. Traces for ingest, the queue, and the consumer. Metrics for queue depth and how long the MST takes to recompute. A dashboard for the disconnect rate.

Why. I can only see Part 2 queue behaviour and Part 3 traversal cost in README curls today. I would want those numbers in operations.

How it fits .NET. The OpenTelemetry .NET SDK, with ASP.NET Core instrumentation, is the usual way to export traces and metrics without rewriting controllers (OpenTelemetry, n.d.). Application Insights is one Azure Monitor exporter for that data (Microsoft, 2025a). I would strip operator notes and attachment paths from spans, and lock down the dashboard. Sampling keeps a high ingest rate affordable. I would record a metric, not a full trace, for every packet. Spans around BFS and the MST would show when those algorithms dominate CPU on a larger mesh (Cormen et al., 2022). I would start with local counters, then export to Azure Monitor without instrumenting the code a second time.

9. Conclusion

I started with telemetry and progressive disclosure. I then built a command and alert pipeline on queues, a stack, a hash set, and a hosted consumer. Part 3 added an AVL history tree, BFS and DFS, and a Prim backbone. The Final PoE is not a separate app. It is the same gateway. The trees, the graph, and the MST are in the APIs and on `/mesh`, and the Part 2 feedback is recorded in the feedback ledger.

Appendix A — Key source map

Concern	Location
Fleet, queues, and mesh store	`SmartX.Api/Storage/GatewayStore.cs`
Background consumer	`SmartX.Api/Storage/TelemetryQueueConsumer.cs`
Mesh seed and down scenario	`SmartX.Api/Simulation/MeshTopologySeeder.cs`, `MeshDownScenarioSeeder.cs`
Mesh APIs	`SmartX.Api/Controllers/MeshController.cs`
AVL history	`SmartX.Shared/Collections/AvlTelemetryHistoryTree.cs`
BFS and reachability	`SmartX.Shared/Mesh/MeshGraphTraverser.cs`, `MeshReachabilityAnalyzer.cs`
Prim MST	`SmartX.Shared/Mesh/MeshMstBuilder.cs`
Topology UI	`SmartX.Client/Pages/Mesh.razor`
Feedback ledger	`docs/FEEDBACK-UPDATES.md`
Operator how-to	`README.md`

Bibliography

Adelson-Velsky, G. and Landis, E. (1962) ‘An algorithm for the organization of information’, *Proceedings of the USSR Academy of Sciences*, 146, pp. 263–266. English translation by M.J. Ricci, *Soviet Mathematics Doklady*, 3, pp. 1259–1263.

Akyildiz, I.F., Wang, X. and Wang, W. (2005) ‘Wireless mesh networks: a survey’, *Computer Networks*, 47(4), pp. 445–487. doi: 10.1016/j.comnet.2004.12.001.

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) *Introduction to algorithms*. 4th edn. Cambridge, MA: MIT Press.

Docker (n.d.-a) *Docker Compose*. Available at: https://docs.docker.com/compose/ (Accessed: 13 November 2026).

Docker (n.d.-b) *Multi-stage builds*. Available at: https://docs.docker.com/build/building/multi-stage/ (Accessed: 13 November 2026).

Espressif Systems (n.d.) *ESP32 Wi-Fi and Bluetooth SoC*. Available at: https://www.espressif.com/en/products/socs/esp32 (Accessed: 13 November 2026).

GitHub (n.d.) *GitHub Actions documentation*. Available at: https://docs.github.com/en/actions (Accessed: 13 November 2026).

Home Assistant (n.d.-a) *MQTT*. Available at: https://www.home-assistant.io/integrations/mqtt/ (Accessed: 13 November 2026).

Home Assistant (n.d.-b) *REST API*. Available at: https://contributors.home-assistant.io/core/internals/rest-api/ (Accessed: 13 November 2026).

Kruskal, J.B. (1956) ‘On the shortest spanning subtree of a graph and the traveling salesman problem’, *Proceedings of the American Mathematical Society*, 7(1), pp. 48–50. doi: 10.1090/S0002-9939-1956-0078686-7.

Kubernetes (n.d.) *Overview*. Available at: https://kubernetes.io/docs/concepts/overview/ (Accessed: 13 November 2026).

Microsoft (2020) *Overview of Entity Framework Core*. Available at: https://learn.microsoft.com/en-us/ef/core/ (Accessed: 13 November 2026).

Microsoft (2023) *Introduction to Semantic Kernel*. Available at: https://learn.microsoft.com/en-us/semantic-kernel/overview/ (Accessed: 13 November 2026).

Microsoft (2025a) *Application Insights OpenTelemetry observability overview*. Available at: https://learn.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview (Accessed: 13 November 2026).

Microsoft (2025b) *ASP.NET Core Blazor*. Available at: https://learn.microsoft.com/en-us/aspnet/core/blazor/?view=aspnetcore-10.0 (Accessed: 13 November 2026).

Microsoft (2025c) *ASP.NET Core Blazor dependency injection*. Available at: https://learn.microsoft.com/en-us/aspnet/core/blazor/fundamentals/dependency-injection?view=aspnetcore-10.0 (Accessed: 13 November 2026).

Microsoft (2025d) *Dictionary<TKey,TValue> class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.dictionary-2?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025e) *HashSet<T> class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.hashset-1?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025f) *JsonStringEnumConverter class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.text.json.serialization.jsonstringenumconverter?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025g) *List<T> class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.list-1?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025h) *Managed identities for Azure resources*. Available at: https://learn.microsoft.com/en-us/entra/identity/managed-identities-azure-resources/overview (Accessed: 13 November 2026).

Microsoft (2025i) *Overview of ASP.NET Core*. Available at: https://learn.microsoft.com/en-us/aspnet/core/overview?view=aspnetcore-10.0 (Accessed: 13 November 2026).

Microsoft (2025j) *PeriodicTimer class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.threading.periodictimer?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025k) *PriorityQueue<TElement,TPriority> class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.priorityqueue-2?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025l) *Queue<T> class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.queue-1?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025m) *SortedDictionary<TKey,TValue> class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.sorteddictionary-2?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2025n) *Stack<T> class*. Available at: https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.stack-1?view=net-10.0 (Accessed: 13 November 2026).

Microsoft (2026a) *Azure Container Apps overview*. Available at: https://learn.microsoft.com/en-us/azure/container-apps/overview (Accessed: 13 November 2026).

Microsoft (2026b) *Azure Key Vault overview*. Available at: https://learn.microsoft.com/en-us/azure/key-vault/general/overview (Accessed: 13 November 2026).

Microsoft (2026c) *Background tasks with hosted services in ASP.NET Core*. Available at: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/host/hosted-services?view=aspnetcore-10.0 (Accessed: 13 November 2026).

Microsoft (2026d) *Call a web API from an ASP.NET Core Blazor app*. Available at: https://learn.microsoft.com/en-us/aspnet/core/blazor/call-web-api?view=aspnetcore-10.0 (Accessed: 13 November 2026).

Microsoft (2026e) *dotnet run command*. Available at: https://learn.microsoft.com/en-us/dotnet/core/tools/dotnet-run (Accessed: 13 November 2026).

Microsoft (2026f) *Enable cross-origin requests (CORS) in ASP.NET Core*. Available at: https://learn.microsoft.com/en-us/aspnet/core/security/cors?view=aspnetcore-10.0 (Accessed: 13 November 2026).

Microsoft (2026g) *How to customize property names and values with System.Text.Json*. Available at: https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/customize-properties (Accessed: 13 November 2026).

Microsoft (2026h) *Azure Cosmos DB overview*. Available at: https://learn.microsoft.com/en-us/azure/cosmos-db/overview (Accessed: 13 November 2026).

Microsoft (2026i) *What is the Azure SQL Database service?* Available at: https://learn.microsoft.com/en-us/azure/azure-sql/database/sql-database-paas-overview?view=azuresql (Accessed: 13 November 2026).

Microsoft (2026j) *What's new in Azure OpenAI in Microsoft Foundry Models (classic)*. Available at: https://learn.microsoft.com/en-us/azure/foundry-classic/openai/whats-new (Accessed: 13 November 2026).

MQTTnet (n.d.) *MQTTnet*. Available at: https://github.com/dotnet/MQTTnet (Accessed: 13 November 2026).

nginx (n.d.) *nginx documentation*. Available at: https://nginx.org/en/docs/ (Accessed: 13 November 2026).

Nielsen, J. (2006) ‘Progressive disclosure’, *Nielsen Norman Group*, 4 December (updated 15 July 2022). Available at: https://www.nngroup.com/articles/progressive-disclosure/ (Accessed: 13 November 2026).

OASIS (2019) *MQTT Version 5.0*. OASIS Standard. Available at: https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html (Accessed: 13 November 2026).

OpenTelemetry (n.d.) *.NET*. Available at: https://opentelemetry.io/docs/languages/dotnet/ (Accessed: 13 November 2026).

Prim, R.C. (1957) ‘Shortest connection networks and some generalizations’, *Bell System Technical Journal*, 36(6), pp. 1389–1401. doi: 10.1002/j.1538-7305.1957.tb01515.x.

Sedgewick, R. and Wayne, K. (2011) *Algorithms*. 4th edn. Addison-Wesley.